

GUÍA DE LABORATORIO N.º 03

Test-Driven Development (TDD) e Integración de TDD en Spring Boot

Curso	Desarrollo Web Integrado
Semana	3
Unidad	Unidad 1: API REST
Tema del sílabo	Test-drive development (TDD). Integración de TDD en Spring.
Modalidad	Presencial
Duración sugerida	3 horas académicas
Herramientas	Visual Studio Code, Java 25, Maven, Spring Boot, JUnit 5, Mockito y Postman
Producto de laboratorio	API REST simple de productos desarrollada con pruebas automatizadas

1. Presentación de la práctica

Contexto curricular. El sílabo del curso ubica la Semana 3 dentro de la Unidad 1: API REST, cuyo logro específico indica que el estudiante implementa una API RESTful de back-end mediante Spring Boot considerando el desarrollo guiado por pruebas. Por ello, esta guía se centra en aplicar TDD de manera práctica sobre una funcionalidad de productos, separando responsabilidades entre modelo, servicio y controlador.

Logro de aprendizaje de la práctica

Al finalizar el laboratorio, el estudiante implementa funcionalidades de una API REST en Spring Boot aplicando el ciclo Red-Green-Refactor, mediante pruebas unitarias con JUnit 5, simulación de dependencias con Mockito y pruebas de controlador con MockMvc.

2. Competencias técnicas que se desarrollan

- Comprender TDD como metodología de diseño y no como una librería.
- Escribir pruebas antes de implementar la lógica de negocio.
- Ejecutar pruebas desde Visual Studio Code y desde Maven.
- Aplicar JUnit 5 para validar resultados esperados, excepciones y reglas de negocio.
- Aplicar Mockito para simular dependencias durante pruebas de controladores.
- Implementar endpoints REST que respondan a pruebas automatizadas.
- Identificar errores frecuentes en pruebas Spring Boot y corregirlos de forma ordenada.

3. Fundamento teórico necesario

3.1. ¿Qué es TDD?

TDD, Test-Driven Development, es una forma de desarrollar software en la que primero se escribe una prueba automatizada que expresa el comportamiento esperado. Luego se implementa el código mínimo para que la prueba pase y, finalmente, se mejora el diseño sin alterar el comportamiento validado.

Fase	Qué se hace	Resultado esperado
Red	Se escribe una prueba que inicialmente falla.	La prueba falla porque la funcionalidad aún no existe o no cumple la regla.
Green	Se implementa el código mínimo necesario.	La prueba pasa.
Refactor	Se mejora el código, nombres, estructura y duplicaciones.	Las pruebas siguen pasando después de mejorar el diseño.

3.2. ¿Qué herramientas se usarán?

Herramienta	Uso en esta práctica
JUnit 5	Permite crear métodos de prueba con @Test y validar resultados con assertions.

AssertJ	Permite escribir aserciones legibles como <code>assertThat(resultado).isEqualTo(...)</code> .
Mockito	Permite crear mocks para aislar el componente que se está probando.
MockMvc	Permite probar controladores REST sin levantar un servidor real.
Spring Boot Starter Test	Incluye las herramientas de prueba más usadas en proyectos Spring Boot.

3.3. Tipos de pruebas que se aplicarán

Tipo de prueba	Qué valida	Ejemplo en el laboratorio
Prueba unitaria	Una clase o método aislado, sin cargar todo Spring.	ProductoServiceTest
Prueba de controlador	La capa web y sus respuestas HTTP.	ProductoControllerTest con <code>@WebMvcTest</code>
Prueba de aplicación	Que el contexto Spring inicia correctamente.	Semana3ApplicationTests

4. Requisitos previos

- Java 25 instalado y configurado en el PATH.
- Apache Maven disponible desde la terminal.
- Visual Studio Code con Extension Pack for Java y Spring Boot Extension Pack.
- Postman o Thunder Client para probar endpoints al final del laboratorio.
- Conocimientos de la Semana 1: estructura Maven y ejecución de Spring Boot.
- Conocimientos de la Semana 2: controladores, endpoints e inyección por constructor.

5. Escenario del laboratorio

Caso práctico. Se construirá una API REST básica para administrar productos. La diferencia respecto a una práctica tradicional es que las funcionalidades se implementarán guiadas por pruebas. El estudiante no debe empezar escribiendo el controlador o el servicio completo; debe expresar primero el comportamiento esperado en una prueba.

Funcionalidad	Regla de negocio
Registrar producto	Debe asignar un ID automático y almacenar el producto.
Validar producto	No se debe aceptar nombre vacío, precio menor o igual a cero ni stock negativo.

Listar productos	Debe devolver todos los productos registrados.
Buscar por ID	Debe devolver el producto si existe o error si no existe.
Exponer API REST	Debe responder con JSON mediante endpoints HTTP.

6. Preparación del proyecto

6.1. Crear o reutilizar el proyecto

Opción recomendada. Puede reutilizar el proyecto de la Semana 2. Si desea iniciar desde cero, cree un proyecto en Spring Initializr con Maven, Java y empaquetado JAR.

Campo	Valor sugerido
Project	Maven
Language	Java
Spring Boot	Última versión estable disponible en Spring Initializr
Packaging	Jar
Java	25
Group	com.utp
Artifact	semana3
Name	semana3

6.2. Dependencias necesarias

- Spring Web: para construir endpoints REST.
- Spring Boot Starter Test: para JUnit 5, Mockito, AssertJ, Spring Test y MockMvc.
- Spring Boot DevTools: opcional, útil durante el desarrollo.

Importante

No agregue JUnit manualmente si el proyecto ya tiene spring-boot-starter-test. Spring Boot Starter Test ya incorpora las herramientas de prueba necesarias para esta práctica.

6.3. Verificar el pom.xml

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
```

```

</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
</dependencies>

```

6.4. Estructura esperada del proyecto

```

semana3
├── pom.xml
├── src
│   ├── main
│   │   ├── java/com/utp/semana3
│   │   │   ├── Semana3Application.java
│   │   │   ├── controller
│   │   │   ├── model
│   │   │   └── service
│   └── test
│       ├── java/com/utp/semana3
│       │   ├── controller
│       │   └── service

```

7. Desarrollo guiado por TDD: capa de servicio

7.1. Crear el modelo mínimo Producto

Decisión didáctica. Para no mezclar TDD con base de datos, esta práctica usa almacenamiento en memoria. La persistencia con MySQL y JPA se abordará en la Unidad 2. Aquí se busca dominar el ciclo de pruebas.

```

package com.utp.semana3.model;

public class Producto {

    private Long id;
    private String nombre;
    private double precio;
    private int stock;

    public Producto() {
    }

    public Producto(Long id, String nombre, double precio, int stock) {
        this.id = id;
        this.nombre = nombre;
        this.precio = precio;
        this.stock = stock;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }
}

```

```

    public double getPrecio() {
        return precio;
    }

    public void setPrecio(double precio) {
        this.precio = precio;
    }

    public int getStock() {
        return stock;
    }

    public void setStock(int stock) {
        this.stock = stock;
    }
}

```

7.2. RED: escribir una prueba que falle

Regla que se desea implementar: cuando se registra un producto válido, el sistema debe asignarle un ID y guardarlo.

```

package com.utp.semana3.service;

import com.utp.semana3.model.Producto;
import org.junit.jupiter.api.Test;

import static org.assertj.core.api.Assertions.assertThat;

class ProductoServiceTest {

    @Test
    void registrarProductoValido_debeAsignarIdYGuardar() {
        ProductoService service = new ProductoService();
        Producto producto = new Producto(null, "Laptop", 3500.00, 10);

        Producto registrado = service.registrar(producto);

        assertThat(registrado.getId()).isNotNull();
        assertThat(registrado.getNombre()).isEqualTo("Laptop");
        assertThat(service.listar()).hasSize(1);
    }
}

```

Resultado esperado en fase RED

La prueba debe fallar o no compilar porque todavía no existe ProductoService o no existen los métodos registrar() y listar(). Este fallo es correcto: demuestra que la prueba se escribió antes de la implementación.

7.3. Ejecutar la prueba en VS Code

1. Abra el archivo ProductoServiceTest.java.
2. Ubique el enlace Run Test encima del método de prueba.
3. Haga clic en Run Test.
4. También puede ejecutar todas las pruebas desde la terminal con mvn test.

```
mvn test
```

7.4. GREEN: implementar el código mínimo

Ahora sí se crea el servicio. El objetivo no es implementar todo el sistema, sino únicamente lo mínimo para que la prueba pase.

```
package com.utp.semana3.service;

import com.utp.semana3.model.Producto;
import org.springframework.stereotype.Service;

import java.util.ArrayList;
import java.util.List;

@Service
public class ProductoService {

    private final List<Producto> productos = new ArrayList<>();
    private long secuencia = 1;

    public Producto registrar(Producto producto) {
        producto.setId(secuencia++);
        productos.add(producto);
        return producto;
    }

    public List<Producto> listar() {
        return productos;
    }
}
```

Resultado esperado en fase GREEN

Al ejecutar nuevamente mvn test, la prueba debe pasar. Si no pasa, revise nombres de paquetes, imports y ubicación del archivo dentro de src/test/java.

7.5. RED: agregar validación de precio

Nueva regla: no se debe registrar un producto con precio menor o igual a cero.

```
@Test
void registrarProductoConPrecioCero_debeLanzarExcepcion() {
    ProductoService service = new ProductoService();
    Producto producto = new Producto(null, "Mouse", 0.00, 5);

    org.assertj.core.api.Assertions.assertThatThrownBy(() -> service.registrar(producto))
        .isInstanceOf(IllegalArgumentException.class)
        .hasMessage("El precio debe ser mayor que cero");
}
```

7.6. GREEN: implementar validación de precio

```
public Producto registrar(Producto producto) {
    if (producto.getPrecio() <= 0) {
        throw new IllegalArgumentException("El precio debe ser mayor que cero");
    }

    producto.setId(secuencia++);
    productos.add(producto);
    return producto;
}
```

7.7. RED: agregar validaciones de nombre y stock

```
@Test
```

```

void registrarProductoSinNombre_debeLanzarExcepcion() {
    ProductoService service = new ProductoService();
    Producto producto = new Producto(null, "", 100.00, 5);

    org.assertj.core.api.Assertions.assertThatThrownBy(() -> service.registrar(producto))
        .isInstanceOf(IllegalArgumentException.class)
        .hasMessage("El nombre es obligatorio");
}

@Test
void registrarProductoConStockNegativo_debeLanzarExcepcion() {
    ProductoService service = new ProductoService();
    Producto producto = new Producto(null, "Teclado", 150.00, -1);

    org.assertj.core.api.Assertions.assertThatThrownBy(() -> service.registrar(producto))
        .isInstanceOf(IllegalArgumentException.class)
        .hasMessage("El stock no puede ser negativo");
}

```

7.8. REFACTOR: mejorar el servicio sin romper pruebas

Ahora que hay varias reglas, conviene extraer la validación a un método privado. Esto es refactorización: el comportamiento no cambia, pero el diseño mejora.

```

package com.utp.semana3.service;

import com.utp.semana3.model.Producto;
import org.springframework.stereotype.Service;

import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

@Service
public class ProductoService {

    private final List<Producto> productos = new ArrayList<>();
    private long secuencia = 1;

    public Producto registrar(Producto producto) {
        validar(producto);
        producto.setId(secuencia++);
        productos.add(producto);
        return producto;
    }

    public List<Producto> listar() {
        return productos;
    }

    public Optional<Producto> buscarPorId(Long id) {
        return productos.stream()
            .filter(producto -> producto.getId().equals(id))
            .findFirst();
    }

    private void validar(Producto producto) {
        if (producto.getNombre() == null || producto.getNombre().isBlank()) {
            throw new IllegalArgumentException("El nombre es obligatorio");
        }

        if (producto.getPrecio() <= 0) {
            throw new IllegalArgumentException("El precio debe ser mayor que cero");
        }

        if (producto.getStock() < 0) {
            throw new IllegalArgumentException("El stock no puede ser negativo");
        }
    }
}

```



```
}
}
```

Punto de control

Ejecute mvn test después de cada refactorización. En TDD no se refactoriza “a ciegas”: se mejora el código con la seguridad de que las pruebas protegen el comportamiento.

8. Desarrollo guiado por TDD: capa web con MockMvc

8.1. ¿Por qué probar el controlador?

El controlador no debe contener la lógica principal del negocio. Su responsabilidad es recibir solicitudes HTTP, delegar al servicio y devolver respuestas correctas. Por ello, se probará que los endpoints respondan con el código HTTP y el JSON esperado.

8.2. RED: prueba para listar productos

```
package com.utp.semana3.controller;

import com.utp.semana3.model.Producto;
import com.utp.semana3.service.ProductoService;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.test.context.bean.override.mockito.MockitoBean;
import org.springframework.test.web.servlet.MockMvc;

import java.util.List;

import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

@WebMvcTest(ProductoController.class)
class ProductoControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockitoBean
    private ProductoService productoService;

    @Test
    void listar_debeRetornarProductosEnJson() throws Exception {
        when(productoService.listar()).thenReturn(List.of(
            new Producto(1L, "Laptop", 3500.00, 10),
            new Producto(2L, "Mouse", 80.00, 20)
        ));

        mockMvc.perform(get("/productos"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.*[0].nombre").value("Laptop"))
            .andExpect(jsonPath("$.*[1].nombre").value("Mouse"));
    }
}
```

Nota sobre @MockitoBean

En versiones modernas de Spring se recomienda @MockitoBean para reemplazar un bean real por un mock dentro

del ApplicationContext de prueba. En proyectos antiguos de Spring Boot 3.3 o anteriores, el docente puede adaptar el ejemplo usando @MockBean.

8.3. GREEN: implementar controlador mínimo

```
package com.utp.semana3.controller;

import com.utp.semana3.model.Producto;
import com.utp.semana3.service.ProductoService;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

@RestController
@RequestMapping("/productos")
public class ProductoController {

    private final ProductoService productoService;

    public ProductoController(ProductoService productoService) {
        this.productoService = productoService;
    }

    @GetMapping
    public List<Producto> listar() {
        return productoService.listar();
    }
}
```

8.4. RED: prueba para registrar producto con POST

```
@Test
void registrar_debeRetornarProductoCreado() throws Exception {
    Producto productoRegistrado = new Producto(1L, "Laptop", 3500.00, 10);

    when(productoService.registrar(org.mockito.ArgumentMatchers.any(Producto.class)))
        .thenReturn(productoRegistrado);

    String json = ""
        {
            "nombre": "Laptop",
            "precio": 3500.00,
            "stock": 10
        }
        """;

    mockMvc.perform(org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post("/productos")
        .contentType(org.springframework.http.MediaType.APPLICATION_JSON)
        .content(json)
        .andExpect(status().isCreated())
        .andExpect(jsonPath("$.id").value(1))
        .andExpect(jsonPath("$.nombre").value("Laptop")));
}
```

8.5. GREEN: implementar endpoint POST

```
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.ResponseStatus;

@PostMapping
```

```
@ResponseStatus(HttpStatus.CREATED)
public Producto registrar(@RequestBody Producto producto) {
    return productoService.registrar(producto);
}
```

8.6. RED: prueba para buscar producto inexistente

```
@Test
void buscarPorIdCuandoNoExiste_debeRetornar404() throws Exception {
    when(productoService.buscarPorId(99L)).thenReturn(java.util.Optional.empty());

    mockMvc.perform(get("/productos/99"))
        .andExpect(status().isNotFound());
}
```

8.7. GREEN: implementar búsqueda por ID

```
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PathVariable;

@GetMapping("/{id}")
public ResponseEntity<Producto> buscarPorId(@PathVariable Long id) {
    return productoService.buscarPorId(id)
        .map(ResponseEntity::ok)
        .orElse(ResponseEntity.notFound().build());
}
```

8.8. Controlador completo esperado

```
package com.utp.semana3.controller;

import com.utp.semana3.model.Producto;
import com.utp.semana3.service.ProductoService;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/productos")
public class ProductoController {

    private final ProductoService productoService;

    public ProductoController(ProductoService productoService) {
        this.productoService = productoService;
    }

    @GetMapping
    public List<Producto> listar() {
        return productoService.listar();
    }

    @GetMapping("/{id}")
    public ResponseEntity<Producto> buscarPorId(@PathVariable Long id) {
        return productoService.buscarPorId(id)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public Producto registrar(@RequestBody Producto producto) {
        return productoService.registrar(producto);
    }
}
```

```
}
}
```

9. Ejecutar y probar la aplicación

9.1. Ejecutar todas las pruebas

```
mvn clean test
```

Resultado esperado

La compilación debe finalizar con BUILD SUCCESS. Si una prueba falla, no continúe agregando nuevas funcionalidades hasta corregir la causa del fallo.

9.2. Ejecutar la aplicación

```
mvn spring-boot:run
```

Cuando la aplicación esté levantada, pruebe los endpoints desde navegador o Postman. Recuerde que el navegador solo permite probar fácilmente solicitudes GET; para POST se recomienda Postman, Thunder Client o curl.

9.3. Probar GET desde navegador

```
http://localhost:8080/productos
```

9.4. Probar POST con Postman

Campo	Valor
Método HTTP	POST
URL	http://localhost:8080/productos
Body	raw / JSON
Header	Content-Type: application/json

```
{
  "nombre": "Laptop",
  "precio": 3500.00,
  "stock": 10
}
```

Respuesta esperada: código HTTP 201 Created y un JSON con id asignado.

10. Buenas prácticas aplicadas

Buena práctica	Aplicación en la guía
----------------	-----------------------

Una responsabilidad por clase	El modelo representa datos, el servicio contiene reglas y el controlador expone HTTP.
Inyección por constructor	ProductoController recibe ProductoService por constructor.
Pruebas antes del código	Cada regla se inicia con una prueba RED.
Servicios sin estado de request	Las variables de método son locales; el almacenamiento en memoria se usa solo con fines didácticos.
Validaciones en la capa de servicio	Las reglas de negocio no se colocan en el controlador.
Mocks en pruebas web	El controlador se prueba sin depender de una implementación real del servicio.

11. Errores frecuentes y soluciones

Error	Causa probable	Solución
No aparece Run Test en VS Code	Falta Extension Pack for Java o el archivo no está en src/test/java.	Instalar extensiones y mover la prueba a la carpeta correcta.
Cannot resolve symbol Test	No está la dependencia spring-boot-starter-test.	Agregar dependencia de pruebas y recargar Maven.
404 al probar /productos	La ruta del controlador no coincide o la aplicación no está ejecutándose.	Revisar @RequestMapping, @GetMapping y consola de Spring Boot.
415 Unsupported Media Type	El POST no envía Content-Type: application/json.	Configurar el header en Postman.
400 Bad Request	JSON mal formado o tipos de datos incorrectos.	Validar llaves, comillas y campos numéricos.
La prueba de controlador no encuentra ProductoService	Falta mock del servicio en @WebMvcTest.	Agregar @MockitoBean private ProductoService productoService;
Se escribió primero todo el código y luego las pruebas	No se aplicó TDD, solo testing posterior.	Rehacer el flujo: prueba fallida, código mínimo, refactorización.

12. Ejercicios complementarios

Ejercicio 1: buscar producto existente

Agregue una prueba para validar que, cuando un producto existe, GET /productos/{id} devuelve código 200 y el JSON del producto.

Ejercicio 2: eliminar producto

Aplique TDD para crear un método eliminar(Long id) en ProductoService y un endpoint DELETE /productos/{id}. Primero escriba la prueba, luego implemente el código mínimo.

Ejercicio 3: actualización de producto

Aplique TDD para crear un endpoint PUT /productos/{id}. Debe validar que no se actualice un producto inexistente y que se mantengan las reglas de precio y stock.

13. Preguntas de reflexión

1. ¿Por qué TDD no es lo mismo que escribir pruebas después de terminar el código?
2. ¿Qué representa la fase RED?
3. ¿Por qué no debe saltarse la fase REFACTOR?
4. ¿Qué diferencia hay entre probar ProductoService y ProductoController?
5. ¿Para qué sirve MockMvc?
6. ¿Por qué en @WebMvcTest se usa un mock del servicio?
7. ¿Qué error se genera cuando Angular o Postman envía un JSON con tipos incorrectos?
8. ¿Cómo ayuda TDD a mejorar el diseño de una API REST?

15. Rúbrica de evaluación

Criterio	Descripción	Puntaje
Aplicación del ciclo TDD	Evidencia pruebas fallidas iniciales, implementación mínima y refactorización.	4
Pruebas unitarias	ProductoServiceTest valida registro, listado y reglas de negocio.	4
Pruebas de controlador	ProductoControllerTest valida respuestas HTTP y JSON usando MockMvc.	4
Implementación funcional	API ejecuta GET y POST correctamente.	3
Calidad del código	Separación por capas, nombres claros e inyección por constructor.	3
Evidencias y explicación	Presenta capturas y reflexión técnica breve.	2
Total		20

16. Cierre conceptual

Idea clave. TDD obliga a pensar primero en el comportamiento esperado antes de programar la solución. En Spring Boot, esta práctica se vuelve especialmente útil porque permite validar la lógica del servicio y la capa REST de forma automática antes de integrar base de datos, seguridad o frontend.

Mensaje final para el estudiante

Un proyecto profesional no se considera confiable solo porque “funciona en mi máquina”. Debe contar con pruebas automatizadas que permitan comprobar que las reglas principales siguen funcionando después de cada cambio.

17. Fuentes consultadas

- Spring Boot Reference Documentation. Testing: <https://docs.spring.io/spring-boot/reference/testing/>
- Spring Boot Reference Documentation. Test Scope Dependencies: <https://docs.spring.io/spring-boot/reference/testing/test-scope-dependencies.html>
- Spring Initializr Reference Guide: <https://docs.spring.io/initializr/docs/current/reference/html/>
- JUnit 5 User Guide: <https://docs.junit.org/5.13.1/user-guide/index.html>
- Spring Framework Reference. @MockitoBean and @MockitoSpyBean: <https://docs.spring.io/spring/reference/testing/annotations/integration-spring/annotation-mockitobean.html>